

Chatbot Whatsapp Berbasis RAG

Artificial Intelligence

Semester Genap 2025-2026

M. Nizar P. Ma'ady, S.Kom., M.IM., M.Kom.

1. Unduh Node.js. Disarankan untuk menginstall versi LTS melalui situs resmi: <https://nodejs.org/>
2. Pastikan telah menyiapkan aplikasi WhatsApp yang siap untuk ditautkan dengan bot.
3. Untuk memastikan Node.js telah berhasil terinstall, cek di console command prompt dengan mengetik perintah berikut:
`node -v`
`npm -v`
4. Buka *search engine* di browser dan akses halaman <https://console.groq.com/keys>, lalu masuk menggunakan akun yang masing-masing.
5. Buat API Key baru (Create API Key) dan beri nama sesuai keinginan Anda. Pilih masa aktif (*expired*) selama 90 days, lalu simpan kode API tersebut dengan baik.
6. Buat file `server-v2.js`

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const fs = require('fs');
const path = require('path');
const { Client, LocalAuth } = require('whatsapp-web.js');
const qrcode = require('qrcode-terminal');
const Groq = require('groq-sdk');

const RAGEngine = require('./lib/rag');
const DatasetManager = require('./lib/dataset');

const app = express();
const PORT = process.env.PORT || 3001;

app.use(cors());
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static('public'));
```

```

const groq = new Groq({
  apiKey: process.env.GROQ_API_KEY
});

const ragEngine = new RAGEngine();
const datasetManager = new DatasetManager();

let client = null;
let qrCodeData = null;
let isReady = false;
let isCleaning = false;
let isInitializing = false;
const handledMessageIds = new Set();

const knowledgeFile = path.join(__dirname, 'knowledge.json');
const behaviorFile = path.join(__dirname, 'config', 'behavior.json');

if (!fs.existsSync(knowledgeFile)) {
  fs.writeFileSync(knowledgeFile, JSON.stringify({ keywords: {},
responses: {} }, null, 2));
}

function loadKnowledge() {
  try {
    const data = fs.readFileSync(knowledgeFile, 'utf8');
    return JSON.parse(data);
  } catch (error) {
    console.error('Error loading knowledge:', error);
    return { keywords: {}, responses: {} };
  }
}

function saveKnowledge(data) {
  try {
    fs.writeFileSync(knowledgeFile, JSON.stringify(data, null, 2));
    ragEngine.clearCache();
    return true;
  } catch (error) {
    console.error('Error saving knowledge:', error);
    return false;
  }
}

function loadBehavior() {
  try {
    if (!fs.existsSync(behaviorFile)) return null;
    const content = fs.readFileSync(behaviorFile, 'utf8');

```

```

        return JSON.parse(content);
    } catch (error) {
        console.error('Error loading behavior config:', error.message);
        return null;
    }
}

function saveBehavior(obj) {
    try {
        fs.mkdirSync(path.dirname(behaviorFile), { recursive: true });
        fs.writeFileSync(behaviorFile, JSON.stringify(obj, null, 2));
        return true;
    } catch (error) {
        console.error('Error saving behavior config:', error.message);
        return false;
    }
}

async function getAIResponse(message, contextItems = [], behavior = null) {
    try {
        const contextBlock = ragEngine.buildContextBlock(contextItems);

        if (!behavior) {
            behavior = loadBehavior() || {
                system_instructions: 'Jawab hanya berdasarkan konteks yang diberikan. Jika tidak ada jawaban, tampilkan fallback.',
                fallback_response: 'Mohon maaf, untuk item itu belum ada di toko kami.',
                max_sentences: 2,
                language: 'id'
            };
        }

        // jika tidak ada konteks yang relevan, kembalikan fallback tanpa memanggil LLM
        if (!contextBlock || contextItems.length === 0) {
            return behavior.fallback_response || 'Mohon maaf, untuk item itu belum ada di toko kami.';
        }

        const systemParts = [];
        if (behavior.system_instructions)
            systemParts.push(behavior.system_instructions);
        systemParts.push(`Jawab hanya menggunakan konteks berikut. Jika konteks tidak memadai, jawab: ${behavior.fallback_response}`);
    }
}

```

```

    systemParts.push(`Jawab maksimal ${behavior.max_sentences || 2}
kalimat. Bahasa: ${behavior.language || 'id'}.`);

    const systemMessage = systemParts.join(' ');
    const userMessage = `Konteks:\n${contextBlock}\n\nPertanyaan:
${message}`;

    const completion = await groq.chat.completions.create({
      messages: [
        { role: 'system', content: systemMessage },
        { role: 'user', content: userMessage }
      ],
      model: process.env.GROQ_MODEL || 'llama-3.1-8b-instant',
      max_tokens: Number(process.env.GROQ_MAX_TOKENS || 200),
      temperature: 0.1
    });

    return completion.choices[0].message.content;
  } catch (error) {
    console.error('Error getting AI response:', error.message);
    return null;
  }
}

async function startBot() {
  if (isReady || isInitializing) {
    return { success: false, message: 'Bot sudah berjalan atau sedang
dimulai' };
  }

  if (isCleaning) {
    return { success: false, message: 'Bot sedang dihentikan, harap
tunggu' };
  }

  isInitializing = true;

  try {
    const clientInstance = initializeClient();
    await clientInstance.initialize();
    isInitializing = false;
    return { success: true, message: 'Bot dimulai, silakan scan QR
code' };
  } catch (error) {
    isInitializing = false;
    client = null;
    qrCodeData = null;
  }
}

```

```

    isCleaning = false;
    throw error;
  }
}

function initializeClient() {
  if (client) return client;

  client = new Client({
    authStrategy: new LocalAuth({ clientId: 'whatsapp-bot' }),
    puppeteer: {
      headless: true,
      args: [
        '--no-sandbox',
        '--disable-setuid-sandbox',
        '--disable-dev-shm-usage',
        '--disable-web-resources',
        '--disable-sync',
        '--disable-translate',
        '--disable-extensions',
        '--disable-default-apps',
        '--disable-component-extensions-with-background-pages'
      ],
      timeout: 120000
    }
  });

  client.on('qr', (qr) => {
    console.log('📄 QR Code Generated');
    console.log('\n🔗 Scan QR Code di bawah untuk connect bot:\n');
    qrCodeData = qr;
    qrcode.generate(qr, { small: true });
    console.log('\n');
  });

  client.on('ready', () => {
    console.log(' Bot is ready!');
    isReady = true;
    isCleaning = false;
  });

  client.on('authenticated', () => {
    console.log(' Client authenticated');
  });

  client.on('disconnected', (reason) => {
    console.log(' Client disconnected:', reason);
  });
}

```

```

    isReady = false;
    client = null;
  });

  const handleIncomingMessage = async (msg, eventName) => {
    try {
      console.log(
        ` ${eventName} event: from=${msg.from}, fromMe=${msg.fromMe},
body=${JSON.stringify(msg.body)} `
      );

      const messageId = msg && msg.id && msg.id._serialized ?
msg.id._serialized : null;
      if (messageId) {
        if (handledMessageIds.has(messageId)) {
          console.log('↩ Ignoring duplicate event for same message');
          return;
        }

        handledMessageIds.add(messageId);
        setTimeout(() => handledMessageIds.delete(messageId), 5 * 60 *
1000);
      }

      if (msg.fromMe) {
        console.log(' Ignoring self-sent message to avoid reply loop');
        return;
      }

      const isPersonalChat = msg.from.endsWith('@c.us') ||
msg.from.endsWith('@lid');
      const isNotStatus = !msg.from.endsWith('@status');
      if (!isPersonalChat || !isNotStatus) {
        console.log(` Ignoring non-personal or status message:
from=${msg.from}`);
        return;
      }

      console.log(` Personal Message from ${msg.from}: ${msg.body}`);
      try {
        await msg.getChat().then(chat => chat.sendStateTyping());
      } catch (e) {
        console.log('Note: Cannot show typing indicator');
      }

      const knowledge = loadKnowledge();
      const keyword = msg.body.toLowerCase().trim();

```

```

    if (knowledge.responses[keyword]) {
      await msg.reply(knowledge.responses[keyword]);
      console.log(' Replied with FAQ keyword match');
    } else {
      const allDocuments = datasetManager.getAllDocuments();
      const contextItems = ragEngine.retrieveContext(
        msg.body,
        allDocuments,
        Number(process.env.RAG_TOP_K || 3)
      );

      console.log(`🔍 RAG Retrieved ${contextItems.length} relevant context(s)`);
      const timeoutPromise = new Promise((_, reject) =>
        setTimeout(() => reject(new Error('AI response timeout')),
15000)
      );

      try {
        const behavior = loadBehavior();
        const aiResponse = await Promise.race([
          getAIResponse(msg.body, contextItems, behavior),
          timeoutPromise
        ]);

        if (aiResponse) {
          await msg.reply(aiResponse);
          console.log(` Replied with AI response (RAG contexts:
${contextItems.length})`);
        } else {
          await msg.reply('Maaf, saya tidak memahami pesan Anda.
Silakan coba lagi.');
```

```

    return client;
}

app.get('/api/bot/status', (req, res) => {
  res.json({
    isReady,
    isCleaning,
    isInitializing,
    hasQRCode: qrCodeData ? true : false
  });
});

app.post('/api/bot/start', async (req, res) => {
  try {
    const result = await startBot();
    return res.json(result);
  } catch (error) {
    console.error('Error starting bot:', error.message);
    res.status(500).json({
      message: 'Error memulai bot. Pastikan koneksi internet stabil dan coba lagi.',
      success: false
    });
  }
});

app.post('/api/bot/stop', async (req, res) => {
  try {
    if (!client) {
      return res.json({ message: 'Bot tidak sedang berjalan', success: false });
    }

    isCleaning = true;
    isReady = false;
    qrCodeData = null;

    const clientToDestroy = client;
    client = null;

    res.json({ message: 'Bot sudah dihentikan', success: true });

    setImmediate(async () => {
      try {
        await clientToDestroy.destroy();
      } catch (destroyError) {

```



```

        console.error('Error destroying client:',
destroyError.message);
    } finally {
        isCleaning = false;
    }
});
} catch (error) {
    console.error('Error stopping bot:', error);
    isCleaning = false;
    res.status(500).json({ message: 'Error menghentikan bot: ' +
error.message, success: false });
}
});

app.get('/api/bot/qr', (req, res) => {
    if (qrCodeData) {
        res.json({ qr: qrCodeData });
    } else {
        res.json({ qr: null });
    }
});

app.get('/api/datasets', (req, res) => {
    res.json({
        datasets: datasetManager.listDatasets(),
        totalDocuments: datasetManager.getAllDocuments().length
    });
});

app.get('/api/datasets/:name', (req, res) => {
    const docs = datasetManager.getDatasetDocuments(req.params.name);
    if (docs.length === 0) {
        return res.status(404).json({ message: 'Dataset tidak ditemukan'
});
    }
    res.json({ documents: docs });
});

app.post('/api/datasets', (req, res) => {
    try {
        const { name, data } = req.body;

        if (!name || !data) {
            return res.status(400).json({ message: 'name dan data harus
diisi' });
        }
    }

```

```

    const result = datasetManager.saveDataset(name, data);
    res.json(result);
  } catch (error) {
    res.status(500).json({ message: 'Error: ' + error.message });
  }
});

app.get('/api/knowledge/keywords', (req, res) => {
  const knowledge = loadKnowledge();
  res.json(knowledge);
});

app.post('/api/knowledge/keyword', (req, res) => {
  try {
    const { keyword, response } = req.body;

    if (!keyword || !response) {
      return res.status(400).json({ message: 'Keyword dan response harus diisi', success: false });
    }

    const knowledge = loadKnowledge();
    knowledge.responses[keyword.toLowerCase().trim()] = response;

    if (saveKnowledge(knowledge)) {
      res.json({ message: 'Keyword berhasil disimpan', success: true });
    } else {
      res.status(500).json({ message: 'Error menyimpan keyword', success: false });
    }
  } catch (error) {
    res.status(500).json({ message: 'Error: ' + error.message, success: false });
  }
});

app.delete('/api/knowledge/keyword/:keyword', (req, res) => {
  try {
    const keyword =
decodeURIComponent(req.params.keyword).toLowerCase();
    const knowledge = loadKnowledge();

    if (knowledge.responses[keyword]) {
      delete knowledge.responses[keyword];
      if (saveKnowledge(knowledge)) {

```

```

        res.json({ message: 'Keyword berhasil dihapus', success: true
    });
    } else {
        res.status(500).json({ message: 'Error menghapus keyword',
    success: false });
    }
    } else {
        res.status(404).json({ message: 'Keyword tidak ditemukan',
    success: false });
    }
    } catch (error) {
        res.status(500).json({ message: 'Error: ' + error.message, success:
    false });
    }
    });

app.get('/api/behavior', (req, res) => {
    try {
        const behavior = loadBehavior();
        if (!behavior) return res.status(404).json({ message: 'Behavior
    config not found' });
        res.json(behavior);
    } catch (error) {
        res.status(500).json({ message: 'Error: ' + error.message });
    }
    });

app.post('/api/behavior', (req, res) => {
    try {
        const obj = req.body;
        if (!obj || typeof obj !== 'object') {
            return res.status(400).json({ message: 'Invalid behavior object'
    });
        }
        const saved = saveBehavior(obj);
        if (saved) return res.json({ message: 'Behavior saved', success:
    true });
        res.status(500).json({ message: 'Error saving behavior', success:
    false });
    } catch (error) {
        res.status(500).json({ message: 'Error: ' + error.message });
    }
    });

app.listen(PORT, () => {
    console.log(` Server berjalan di 

```

```

    console.log(` Datasets loaded:
    ${datasetManager.listDatasets().length}`);

    if (process.env.AUTO_START_BOT !== 'false') {
      setTimeout(() => {
        startBot().catch(error => {
          console.error('Error auto-starting bot:', error.message);
        });
      }, 500);
    }
  });

```

7. Lib/rag.js

```

const STOPWORDS_ID = new Set([
  'yang', 'dan', 'di', 'ke', 'dari', 'untuk', 'dengan', 'atau', 'pada',
  'adalah',
  'ini', 'itu', 'dalam', 'juga', 'karena', 'agar', 'sebagai', 'saat',
  'oleh', 'akan',
  'bisa', 'dapat', 'sudah', 'belum', 'kami', 'kamu', 'anda', 'saya',
  'aku', 'kita',
  'mereka', 'apa', 'siapa', 'kapan', 'dimana', 'bagaimana', 'kenapa',
  'jika', 'kalau'
]);

class RAGEngine {
  constructor() {
    this.cache = {
      signature: '',
      index: null
    };
  }

  tokenize(text) {
    if (!text) return [];
    return text
      .toLowerCase()
      .replace(/[^a-z0-9\s]/g, ' ')
      .split(/\s+/)
      .filter(token => token.length > 2 && !STOPWORDS_ID.has(token));
  }

  splitIntoChunks(text, chunkSize = 700, overlap = 120) {
    if (!text) return [];

    const normalized = text.replace(/\r/g, '').trim();
    if (!normalized) return [];
  }

```

```

const chunks = [];
let start = 0;

while (start < normalized.length) {
  let end = Math.min(start + chunkSize, normalized.length);

  if (end < normalized.length) {
    const lastBreak = normalized.lastIndexOf('\n', end);
    if (lastBreak > start + 120) {
      end = lastBreak;
    }
  }

  const chunk = normalized.slice(start, end).trim();
  if (chunk.length > 40) {
    chunks.push(chunk);
  }

  if (end >= normalized.length) break;
  start = Math.max(end - overlap, start + 1);
}

return chunks;
}

buildTfMap(tokens) {
  const tf = new Map();
  for (const token of tokens) {
    tf.set(token, (tf.get(token) || 0) + 1);
  }
  return tf;
}

buildRagIndex(documents) {
  if (!documents || documents.length === 0) {
    return { idf: new Map(), vectors: [] };
  }

  const tokenizedDocs = documents.map(doc =>
this.tokenize(doc.text));
  const docFreq = new Map();

  tokenizedDocs.forEach(tokens => {
    const uniqueTokens = new Set(tokens);
    uniqueTokens.forEach(token => {
      docFreq.set(token, (docFreq.get(token) || 0) + 1);
    });
  });
}

```

```

    });
  });

  const totalDocs = Math.max(documents.length, 1);
  const idf = new Map();

  docFreq.forEach((freq, token) => {
    idf.set(token, Math.log((totalDocs + 1) / (freq + 1)) + 1);
  });

  const vectors = tokenizedDocs.map((tokens, idx) => {
    const tf = this.buildTfMap(tokens);
    const vector = new Map();
    let normSquared = 0;

    tf.forEach((count, token) => {
      const weight = count * (idf.get(token) || 0);
      vector.set(token, weight);
      normSquared += weight * weight;
    });

    return {
      source: documents[idx].source,
      text: documents[idx].text,
      vector,
      norm: Math.sqrt(normSquared)
    };
  });

  return { idf, vectors };
}

retrieveContext(query, documents, topK = 3) {
  if (!documents || documents.length === 0) return [];

  const { idf, vectors } = this.buildRagIndex(documents);
  if (!vectors.length) return [];

  const queryTokens = this.tokenize(query);
  if (!queryTokens.length) return [];

  const queryTf = this.buildTfMap(queryTokens);
  const queryVector = new Map();
  let queryNormSquared = 0;

  queryTf.forEach((count, token) => {

```

```

    const weight = count * (idf.get(token) || 0);
    if (weight > 0) {
      queryVector.set(token, weight);
      queryNormSquared += weight * weight;
    }
  });

  const queryNorm = Math.sqrt(queryNormSquared);
  if (!queryNorm) return [];

  const scored = vectors
    .map(item => {
      if (!item.norm) return { ...item, score: 0 };

      let dot = 0;
      queryVector.forEach((qWeight, token) => {
        const dWeight = item.vector.get(token);
        if (dWeight) dot += qWeight * dWeight;
      });

      return {
        source: item.source,
        text: item.text,
        score: dot / (queryNorm * item.norm)
      };
    })
    .filter(item => item.score > 0.07)
    .sort((a, b) => b.score - a.score)
    .slice(0, topK);

  return scored;
}

buildContextBlock(contextItems) {
  if (!contextItems || !contextItems.length) return '';

  return contextItems
    .map((item, idx) => {
      const cleanText = item.text.replace(/\s+/g, ' ').trim();
      return `[Konteks ${idx + 1}] Sumber:
${item.source}\n${cleanText}`;
    })
    .join('\n\n');
}

clearCache() {
  this.cache.signature = '';
}

```

```

    this.cache.index = null;
  }
}

module.exports = RAGEngine;

```

8. Lib/dataset.js

```

const fs = require('fs');
const path = require('path');

class DatasetManager {
  constructor() {
    this.dataDir = path.join(__dirname, '..', 'data');
    this.ensureDataDir();
    this.datasets = new Map();
    this.loadAllDatasets();
  }

  parseCsvLine(line) {
    const values = [];
    let current = '';
    let insideQuotes = false;

    for (let index = 0; index < line.length; index += 1) {
      const character = line[index];

      if (character === '"') {
        if (insideQuotes && line[index + 1] === '"') {
          current += '"';
          index += 1;
        } else {
          insideQuotes = !insideQuotes;
        }
        continue;
      }

      if (character === ',' && !insideQuotes) {
        values.push(current.trim());
        current = '';
        continue;
      }

      current += character;
    }

    values.push(current.trim());
  }

```



```

    return values;
  }

  buildTextFromCsvRow(headers, row) {
    const fields = [];

    headers.forEach((header, index) => {
      const value = row[index] ? row[index].trim() : '';
      if (!value) return;

      const normalizedHeader = header.toLowerCase();
      if (normalizedHeader.includes('href') ||
normalizedHeader.includes('src')) return;

      fields.push(`${header}: ${value}`);
    });

    return fields.join('\n');
  }

  extractCsvLabel(row) {
    for (const value of row) {
      const cleanValue = value ? value.trim() : '';
      if (!cleanValue) continue;
      if (/^https?:\/\/i.test(cleanValue)) continue;
      if (/^data:/i.test(cleanValue)) continue;
      if (/^[^d.,%+~]+$/i.test(cleanValue)) continue;
      if (cleanValue.length < 4) continue;
      return cleanValue;
    }

    return 'baris';
  }

  loadCsvDataset(filePath, datasetName) {
    const content = fs.readFileSync(filePath,
'utf8').replace(/^uFEFF/, '');
    const lines = content
      .split(/\r?\n/)
      .map(line => line.trim())
      .filter(line => line.length > 0);

    if (lines.length < 2) {
      return {
        name: datasetName,
        file: filePath,
        data: { documents: [] },

```

```

        loadedAt: new Date().toISOString()
    };
}

const headers = this.parseCsvLine(lines[0]);
const documents = [];

for (let index = 1; index < lines.length; index += 1) {
    const row = this.parseCsvLine(lines[index]);
    const text = this.buildTextFromCsvRow(headers, row);

    if (!text) continue;

    const title = this.extractCsvLabel(row);
    documents.push({
        source: `${datasetName}/${title}`,
        text
    });
}

return {
    name: datasetName,
    file: filePath,
    data: {
        metadata: {
            name: datasetName,
            type: 'csv'
        },
        documents
    },
    loadedAt: new Date().toISOString()
};
}

ensureDataDir() {
    if (!fs.existsSync(this.dataDir)) {
        fs.mkdirSync(this.dataDir, { recursive: true });
        console.log(`📁 Created data directory: ${this.dataDir}`);
    }
}

loadAllDatasets() {
    try {
        const files = fs.readdirSync(this.dataDir);

        for (const file of files) {
            const filePath = path.join(this.dataDir, file);

```

```

    try {
      if (file.endsWith('.json')) {
        const datasetName = file.replace('.json', '');
        const content = fs.readFileSync(filePath, 'utf8');
        const data = JSON.parse(content);
        this.datasets.set(datasetName, {
          name: datasetName,
          file: filePath,
          data: data,
          loadedAt: new Date().toISOString()
        });
        console.log(`✅ Loaded dataset: ${datasetName}`);
      }

      if (file.endsWith('.csv')) {
        const datasetName = file.replace('.csv', '');
        const dataset = this.loadCsvDataset(filePath,
datasetName);
        this.datasets.set(datasetName, dataset);
        console.log(`✅ Loaded CSV dataset: ${datasetName}`);
      }
    } catch (error) {
      console.error(`❌ Error loading dataset ${file}:`,
error.message);
    }
  }

  if (this.datasets.size === 0) {
    console.log('⚠️ No datasets found in data directory');
  }
} catch (error) {
  console.error('Error loading datasets:', error.message);
}
}

getAllDocuments() {
  const allDocs = [];

  for (const [name, dataset] of this.datasets) {
    if (dataset.data.documents &&
Array.isArray(dataset.data.documents)) {
      for (const doc of dataset.data.documents) {
        allDocs.push({
          source: `${name}/${doc.source || 'unknown'}`,
          text: doc.text || ''

```

```

    });
  }
}

if (dataset.data.faq && Array.isArray(dataset.data.faq)) {
  for (const faq of dataset.data.faq) {
    allDocs.push({
      source: `${name}/FAQ: ${faq.question || 'unknown'}`,
      text: `${faq.question}\n${faq.answer}`
    });
  }
}

return allDocs;
}

getDatasetDocuments(datasetName) {
  const dataset = this.datasets.get(datasetName);
  if (!dataset) return [];

  const docs = [];

  if (dataset.data.documents &&
Array.isArray(dataset.data.documents)) {
    for (const doc of dataset.data.documents) {
      docs.push({
        source: `${datasetName}/${doc.source || 'unknown'}`,
        text: doc.text || ''
      });
    }
  }

  if (dataset.data.faq && Array.isArray(dataset.data.faq)) {
    for (const faq of dataset.data.faq) {
      docs.push({
        source: `${datasetName}/FAQ: ${faq.question || 'unknown'}`,
        text: `${faq.question}\n${faq.answer}`
      });
    }
  }

  return docs;
}

saveDataset(datasetName, data) {
  try {

```

```

        const filePath = path.join(this.dataDir,
`_${datasetName}.json`);
        fs.writeFileSync(filePath, JSON.stringify(data, null, 2));

        this.datasets.set(datasetName, {
            name: datasetName,
            file: filePath,
            data: data,
            loadedAt: new Date().toISOString()
        });

        console.log(`✅ Saved dataset: ${datasetName}`);
        return { success: true, message: `Dataset ${datasetName} saved`
    };
    } catch (error) {
        console.error(`Error saving dataset ${datasetName}:`,
error.message);
        return { success: false, message: error.message };
    }
}

listDatasets() {
    return Array.from(this.datasets.values()).map(d => ({
        name: d.name,
        loadedAt: d.loadedAt,
        documentCount: this.getDatasetDocuments(d.name).length
    }));
}

reloadDatasets() {
    this.datasets.clear();
    this.loadAllDatasets();
}
}

module.exports = DatasetManager;

```

9. Public/index.html

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>WhatsApp Bot Simple</title>
    <style>

```

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
Roboto, sans-serif;
  background: linear-gradient(135deg, #f5f7fa 0%, #eef2f5
100%);
  color: #2c3e50;
  padding: 24px;
  min-height: 100vh;
}

.container {
  max-width: 1000px;
  margin: 0 auto;
}

.header {
  background: white;
  padding: 32px;
  border-radius: 12px;
  margin-bottom: 24px;
  box-shadow: 0 2px 8px rgba(44, 62, 80, 0.08);
  border-left: 5px solid #2c3e50;
}

.header h1 {
  font-size: 28px;
  margin-bottom: 16px;
  font-weight: 700;
}

.status-bar {
  display: flex;
  align-items: center;
  gap: 12px;
  font-size: 14px;
}

.status-dot {
  width: 12px;
  height: 12px;
  border-radius: 50%;
}
```

```
        background: #bdc3c7;
    }

    .status-dot.active {
        background: #1abc9c;
        box-shadow: 0 0 8px rgba(26, 188, 156, 0.4);
    }

    .tabs {
        display: flex;
        gap: 8px;
        background: white;
        padding: 0 32px;
        border-bottom: 1px solid #ecf0f1;
        border-radius: 12px 12px 0 0;
        box-shadow: 0 2px 8px rgba(44, 62, 80, 0.08);
    }

    .tab {
        background: transparent;
        border: none;
        padding: 16px 24px;
        cursor: pointer;
        font-size: 14px;
        font-weight: 600;
        color: #7f8c8d;
        border-bottom: 3px solid transparent;
        transition: all 0.3s;
    }

    .tab:hover {
        color: #2c3e50;
        background: rgba(44, 62, 80, 0.04);
    }

    .tab.active {
        color: #2c3e50;
        border-bottom-color: #2c3e50;
    }

    .content {
        background: white;
        padding: 40px;
        border-radius: 0 12px 12px 12px;
        box-shadow: 0 2px 8px rgba(44, 62, 80, 0.08);
        min-height: 400px;
    }
}
```

```
.tab-pane {
  display: none;
}

.tab-pane.active {
  display: block;
}

h2 {
  font-size: 24px;
  margin-bottom: 8px;
  font-weight: 700;
}

.subtitle {
  font-size: 14px;
  color: #7f8c8d;
  margin-bottom: 24px;
}

.form-group {
  margin-bottom: 24px;
}

label {
  display: block;
  margin-bottom: 8px;
  font-weight: 600;
  color: #2c3e50;
  font-size: 14px;
}

input, textarea {
  width: 100%;
  padding: 12px 14px;
  border: 1.5px solid #e0e0e0;
  border-radius: 8px;
  font-size: 14px;
  font-family: inherit;
  transition: all 0.3s;
  background: #fafbfc;
}

input:focus, textarea:focus {
  outline: none;
  border-color: #2c3e50;
}
```



```
        background: white;
        box-shadow: 0 0 0 3px rgba(44, 62, 80, 0.08);
    }

    textarea {
        min-height: 100px;
        resize: vertical;
    }

    .btn {
        background: #2c3e50;
        color: white;
        padding: 11px 26px;
        border: none;
        border-radius: 8px;
        cursor: pointer;
        font-size: 14px;
        font-weight: 600;
        transition: all 0.3s;
        box-shadow: 0 2px 6px rgba(44, 62, 80, 0.15);
    }

    .btn:hover {
        background: #1a252f;
        box-shadow: 0 4px 12px rgba(44, 62, 80, 0.25);
        transform: translateY(-1px);
    }

    .btn-danger {
        background: #e74c3c;
    }

    .btn-danger:hover {
        background: #c0392b;
    }

    .btn-success {
        background: #1abc9c;
    }

    .btn-success:hover {
        background: #16a085;
    }

    .keyword-list {
        margin-top: 32px;
    }
```

```
.keyword-item {
  background: #f8fafb;
  padding: 18px;
  margin-bottom: 12px;
  border-radius: 8px;
  display: flex;
  justify-content: space-between;
  align-items: start;
  gap: 16px;
  border: 1px solid #ecf0f1;
  transition: all 0.3s;
}

.keyword-item:hover {
  background: white;
  border-color: #d0d5db;
  box-shadow: 0 4px 12px rgba(44, 62, 80, 0.1);
}

.keyword-info {
  flex: 1;
}

.keyword-info strong {
  display: block;
  margin-bottom: 6px;
  font-size: 15px;
  font-weight: 700;
}

.keyword-info p {
  color: #556983;
  font-size: 13px;
  line-height: 1.6;
  white-space: pre-wrap;
  word-break: break-word;
}

.keyword-actions {
  display: flex;
  gap: 8px;
}

.keyword-actions .btn {
  padding: 8px 16px;
  font-size: 12px;
}
```

```
}

.alert {
  padding: 16px;
  border-radius: 8px;
  margin-bottom: 24px;
  font-size: 14px;
  border-left: 4px solid;
  line-height: 1.6;
}

.alert-info {
  background: #e8f4f8;
  color: #0a4f6b;
  border-left-color: #3498db;
}

.qr-container {
  text-align: center;
  padding: 40px 20px;
}

.qr-container h3 {
  margin-bottom: 12px;
  font-size: 18px;
}

.qr-container p {
  color: #7f8c8d;
  margin-bottom: 24px;
  font-size: 13px;
  line-height: 1.6;
}

#qrcode {
  max-width: 300px;
  margin: 20px auto;
  padding: 16px;
  background: #f8fafb;
  border-radius: 12px;
  border: 1px solid #ecf0f1;
}

.ready-section {
  text-align: center;
  padding: 40px;
}
```

```
.ready-section h3 {
  font-size: 18px;
  margin-bottom: 12px;
  color: #1abc9c;
  font-weight: 700;
}

.ready-section p {
  color: #7f8c8d;
  font-size: 14px;
}

/* Toast Notification */
.notification {
  position: fixed;
  bottom: -100px;
  right: 20px;
  padding: 16px 24px;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 500;
  box-shadow: 0 4px 12px rgba(44, 62, 80, 0.2);
  transition: all 0.3s;
  max-width: 350px;
  z-index: 9999;
}

.notification.show {
  bottom: 20px;
}

.notification-success {
  background: #1abc9c;
  color: white;
}

.notification-error {
  background: #e74c3c;
  color: white;
}

.notification-warning {
  background: #f39c12;
  color: white;
}
```

```

        .notification-info {
            background: #3498db;
            color: white;
        }

        .button-group {
            display: flex;
            gap: 12px;
        }

        @media (max-width: 768px) {
            .header {
                padding: 20px;
            }

            .tabs {
                padding: 0 12px;
            }

            .content {
                padding: 24px;
            }

            .keyword-item {
                flex-direction: column;
            }

            .keyword-actions {
                width: 100%;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <!-- Header -->
        <div class="header">
            <h1>Bot WhatsApp Sederhana</h1>
            <div class="status-bar">
                <span class="status-dot" id="statusDot"></span>
                <span id="statusText">Memeriksa status...</span>
                <div style="margin-left: auto; display: flex; gap:
12px;">
                    <button class="btn" id="startBtn" style="display:
none;">Mulai Bot</button>
                    <button class="btn btn-danger" id="stopBtn"
style="display: none;">Hentikan Bot</button>

```

```

        </div>
    </div>
</div>

<div class="tabs">
    <button class="tab active"
onclick="switchTab('control')">Kontrol Bot</button>
    <button class="tab" onclick="switchTab('knowledge')">Basis
Pengetahuan</button>
</div>

<!-- Content -->
<div class="content">
    <div id="control" class="tab-pane active">
        <h2>Kontrol Bot</h2>
        <p class="subtitle">Kelola koneksi bot WhatsApp
Anda</p>

        <div class="alert alert-info">
            Gunakan panel ini untuk menghubungkan bot dengan
akun WhatsApp Anda
        </div>

        <div id="qrSection" style="display: none;">
            <div class="qr-container">
                <h3>Pindai Kode QR</h3>
                <p>Buka WhatsApp di HP Anda → Menu → Perangkat
Tertaut → Tautkan Perangkat</p>
                <div id="qrcode"></div>
            </div>
        </div>

        <div id="readySection" style="display: none;">
            <div class="ready-section">
                <h3>✓ Bot Terhubung</h3>
                <p>Bot siap menerima dan membalas pesan
WhatsApp</p>
            </div>
        </div>
    </div>

    <div id="knowledge" class="tab-pane">
        <h2>Basis Pengetahuan</h2>
        <p class="subtitle">Kelola kata kunci dan respons
otomatis</p>

        <div class="form-group">
            <label>Kata Kunci (huruf kecil)</label>

```

```

        <input type="text" id="keyword" placeholder="cth:
halo, menu, bantuan">
    </div>

    <div class="form-group">
        <label>Respons</label>
        <textarea id="response" placeholder="Tuliskan
respons otomatis..."></textarea>
    </div>

    <div class="button-group">
        <button class="btn btn-success"
onclick="saveKeyword()">Simpan Kata Kunci</button>
        <button class="btn"
onclick="clearForm()">Bersihkan</button>
    </div>

    <div class="keyword-list">
        <h3>Daftar Kata Kunci</h3>
        <div id="keywordItems"></div>
    </div>
</div>
</div>
</div>

<script src="app.js"></script>
</body>
</html>

```

10. Public/app.js

```

const API_URL = '/api';

function showNotification(message, type = 'info', duration = 3000) {
    const notification = document.createElement('div');
    notification.className = `notification notification-${type}`;
    notification.textContent = message;

    document.body.appendChild(notification);
    setTimeout(() => notification.classList.add('show'), 10);

    setTimeout(() => {
        notification.classList.remove('show');
        setTimeout(() => notification.remove(), 300);
    }, duration);
}

```

```

function switchTab(tab) {
    document.querySelectorAll('.tab').forEach(t =>
t.classList.remove('active'));
    event.target.classList.add('active');

    document.querySelectorAll('.tab-pane').forEach(c =>
c.classList.remove('active'));
    document.getElementById(tab).classList.add('active');

    if (tab === 'knowledge') {
        loadKeywords();
    }
}

async function checkBotStatus() {
    try {
        const response = await fetch(`${API_URL}/bot/status`);
        const data = await response.json();

        const statusDot = document.getElementById('statusDot');
        const statusText = document.getElementById('statusText');
        const startBtn = document.getElementById('startBtn');
        const stopBtn = document.getElementById('stopBtn');
        const qrSection = document.getElementById('qrSection');
        const readySection = document.getElementById('readySection');

        if (data.isReady) {
            statusDot.classList.add('active');
            statusText.textContent = 'Bot Connected';
            startBtn.style.display = 'none';
            stopBtn.style.display = 'inline-block';
            qrSection.style.display = 'none';
            readySection.style.display = 'block';
        } else if (data.hasQRCode) {
            statusDot.classList.remove('active');
            statusText.textContent = 'Waiting for QR Scan';
            startBtn.style.display = 'none';
            stopBtn.style.display = 'inline-block';
            qrSection.style.display = 'block';
            readySection.style.display = 'none';
            loadQRCode();
        } else if (data.isInitializing) {
            statusDot.classList.remove('active');
            statusText.textContent = 'Initializing...';
            startBtn.style.display = 'none';
            stopBtn.style.display = 'none';
            qrSection.style.display = 'none';
        }
    }
}

```



```

        readySection.style.display = 'none';
    } else {
        statusDot.classList.remove('active');
        statusText.textContent = 'Bot Offline';
        startBtn.style.display = 'inline-block';
        stopBtn.style.display = 'none';
        qrSection.style.display = 'none';
        readySection.style.display = 'none';
    }
} catch (error) {
    console.error('Error checking status:', error);
}
}

async function loadQRCode() {
    try {
        const response = await fetch(`${API_URL}/bot/qr`);
        const data = await response.json();

        if (data.qr) {
            const qrContainer = document.getElementById('qrcode');
            qrContainer.innerHTML = '';

            const qrImage = document.createElement('img');
            qrImage.src = `https://api.qrserver.com/v1/create-qr-code/?size=300x300&data=${encodeURIComponent(data.qr)}`;
            qrImage.style.width = '280px';
            qrImage.style.height = '280px';
            qrContainer.appendChild(qrImage);
        }
    } catch (error) {
        console.error('Error loading QR:', error);
    }
}

document.getElementById('startBtn').addEventListener('click', async () => {
    try {
        const response = await fetch(`${API_URL}/bot/start`, { method: 'POST' });
        const data = await response.json();
        showNotification(data.message, 'success');

        checkBotStatus();
        const interval = setInterval(checkBotStatus, 2000);

        setTimeout(() => clearInterval(interval), 120000);
    }
});

```

```

    } catch (error) {
        showNotification('Error: ' + error.message, 'error');
    }
});

document.getElementById('stopBtn').addEventListener('click', async ()
=> {
    try {
        document.getElementById('stopBtn').disabled = true;
        const response = await fetch(`${API_URL}/bot/stop`, { method:
'POST' });
        const data = await response.json();
        showNotification(data.message, 'success');

document.getElementById('statusDot').classList.remove('active');
        document.getElementById('statusText').textContent = 'Bot
Offline';
        document.getElementById('qrSection').style.display = 'none';
        document.getElementById('readySection').style.display = 'none';
        document.getElementById('startBtn').style.display = 'inline-
block';
        document.getElementById('stopBtn').style.display = 'none';
        document.getElementById('stopBtn').disabled = false;
    } catch (error) {
        showNotification('Error: ' + error.message, 'error');
        document.getElementById('stopBtn').disabled = false;
    }
});

async function loadKeywords() {
    try {
        const response = await fetch(`${API_URL}/knowledge/keywords`);
        const data = await response.json();

        const container = document.getElementById('keywordItems');
        container.innerHTML = '';

        if (Object.keys(data.responses).length === 0) {
            container.innerHTML = '<p style="color: #999; text-align:
center; padding: 40px;">Belum ada keyword. Tambahkan keyword baru di
atas!</p>';
            return;
        }

        Object.entries(data.responses).forEach(([keyword, response]) =>
{

```

```

        const item = document.createElement('div');
        item.className = 'keyword-item';
        item.innerHTML = `
            <div class="keyword-info">
                <strong>${keyword}</strong>
                <p>${response}</p>
            </div>
            <div class="keyword-actions">
                <button class="btn"
onclick="editKeyword('${keyword}')">Edit</button>
                <button class="btn btn-danger"
onclick="deleteKeyword('${keyword}')">Delete</button>
            </div>
        `;
        container.appendChild(item);
    });
} catch (error) {
    console.error('Error loading keywords:', error);
}
}

async function saveKeyword() {
    const keyword =
document.getElementById('keyword').value.trim().toLowerCase();
    const response = document.getElementById('response').value.trim();

    if (!keyword || !response) {
        showNotification('Keyword dan response harus diisi!',
'warning');
        return;
    }

    try {
        const res = await fetch(`${API_URL}/knowledge/keyword`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ keyword, response })
        });

        const data = await res.json();
        showNotification(data.message, data.success ? 'success' :
'error');

        if (data.success) {
            clearForm();
            loadKeywords();
        }
    }

```

```

    } catch (error) {
        showNotification('Error: ' + error.message, 'error');
    }
}

function editKeyword(keyword) {
    document.getElementById('keyword').value = keyword;
    loadKeywordsForEdit(keyword);
    document.getElementById('keyword').focus();
}

async function loadKeywordsForEdit(keyword) {
    try {
        const response = await fetch(`${API_URL}/knowledge/keywords`);
        const data = await response.json();

        if (data.responses[keyword]) {
            document.getElementById('response').value =
data.responses[keyword];
        }
    } catch (error) {
        console.error('Error:', error);
    }
}

async function deleteKeyword(keyword) {
    if (!confirm(`Hapus kata kunci "${keyword}"?`)) return;

    try {
        const response = await
fetch(`${API_URL}/knowledge/keyword/${encodeURIComponent(keyword)}`, {
            method: 'DELETE'
        });

        const data = await response.json();
        showNotification(data.message, data.success ? 'success' :
'error');

        if (data.success) {
            loadKeywords();
        }
    } catch (error) {
        showNotification('Error: ' + error.message, 'error');
    }
}

function clearForm() {

```

```

    document.getElementById('keyword').value = '';
    document.getElementById('response').value = '';
    document.getElementById('keyword').focus();
}

checkBotStatus();
setInterval(checkBotStatus, 5000);

```

11. Data/

Download di link berikut:

<https://drive.google.com/file/d/1pHKHm39eKEDlp4cIOnzeahnTngT0eZSs/view?usp=sharing>

12. Config/behavior.json

```

{
  "language": "id",
  "max_sentences": 2,
  "system_instructions": " Anda adalah asisten untuk toko CV Netafarm yang hanya boleh menjawab berdasarkan konteks yang diberikan dari basis data. Jangan menambahkan informasi di luar konteks. Jika jawaban tidak ada dalam konteks, jawab dengan frasa fallback yang disediakan.",
  "fallback_response": "Mohon maaf, untuk item itu belum ada di toko kami.",
  "reply_style": "singkat"
}

```

13. Paste ini untuk memunculkan package.json:

npm init -y

```

"scripts": {
  "start": "node server-v2.js",
  "dev": "nodemon server-v2.js"
},
"keywords": ["whatsapp", "bot", "groq", "ai"],
"author": "",
"license": "MIT",
"dependencies": {
  "express": "^4.18.2",
  "cors": "^2.8.5",
  "body-parser": "^1.20.2",
  "dotenv": "^17.0.0",
  "groq-sdk": "^0.5.0",
  "whatsapp-web.js": "^1.25.0",
  "qrcode-terminal": "^0.12.0"
},
"devDependencies": {

```

```
"nodemon": "^3.0.1"  
}  
}
```

14. Paste di terminal:

npm install untuk menginstall package yang ada di package.json

npm start

Tugas kelompok di kelas

1. Melakukan scraping data produk dari Shopee dan menambahkannya ke dalam folder dataset agar dapat digunakan sebagai dataset chatbot berbasis RAG.

Panduan pengerjaan dapat dilihat pada tautan berikut:

<https://drive.google.com/file/d/1KLfOIETNNjt7hi4F1Oqd46G5ehvJvDBk/view?usp=sharing>

2. Mengubah perilaku chatbot dengan membatasi respons/chat pada LLM sesuai kebutuhan (Contoh: nama toko, lokasi, dan batasi pertanyaan di luar toko).
3. Presentasi perwakilan kelompok, jika sudah boleh pulang.

Tugas Kelompok di luar kelas

1. Melakukan scraping data dari toko di Shopee yang memiliki banyak kategori produk.
 - Setiap kategori minimal terdiri dari 10 produk, dan jumlah kategori disesuaikan dengan jumlah anggota kelompok.
 - Contoh: jika anggota kelompok berjumlah 5 orang, maka harus terdapat 5 kategori produk yang berbeda.
 - Tugas dikerjakan secara individu dengan pembagian kategori antar anggota kelompok.
2. Membuat halaman welcome page yang menampilkan nama toko serta tombol yang menuju ke WhatsApp untuk setiap kategori produk.
3. Deploy 'welcome page' ke vercel.
 - Deploy dilakukan oleh perwakilan kelompok.
 - Pembuatan chatbot WhatsApp tetap dikerjakan secara individu.